NotebookLM / Gemini Notebook Clone

Complete setup, implementation, evaluation, deployment, and submission roadmap

BUILD	VERIFY	DEPLOY	SUBMIT
Notebooks, sources, RAG chat, citations, artifacts	Tests, retrieval methods, persistence	GitHub Actions + Render + secrets	Repo, live URL, demo video, architecture

What is already in the ZIP

- A Gradio web app with notebook CRUD and notebook-specific data.
- PDF, PPTX, TXT, Markdown, and web ingestion with chunking and embeddings.
- Persistent SQLite vector retrieval, Gemini answers, citations, and saved chat history.
- Downloadable report and quiz artifacts, including a quiz answer key.
- Similarity-versus-MMR evaluation, tests, architecture, Docker, Render, and CI/CD.

Important: the files provide a complete implementation, but you must add your own Gemini key, GitHub URL, deployed URL, measured evaluation results, and personal demonstration recording. Those cannot be truthfully pre-created for you.

1. Run the project locally

Unzip the project, open its folder in VS Code, and open a terminal at the folder containing **app.py**.

```
python -m venv .venv
# Windows PowerShell
.venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
# Windows
Copy-Item .env.example .env
# macOS/Linux: cp .env.example .env
python app.py
```

Open **http://localhost:7860**. If PowerShell blocks activation, run `Set-ExecutionPolicy -Scope Process Bypass` and activate again.

Add Gemini safely

Get a Gemini API key from Google AI Studio. In **.env**, replace the placeholder after `GEMINI_API_KEY=`. Never paste the key into source code, screenshots, GitHub, or your video.

First functional test

1. Create a notebook named RAG Study.
2. Upload sample_sources/intro_to_rag.txt.
3. Ask: How does RAG work?
4. Confirm bracketed citations and retrieved source references.
5. Generate both report and quiz, then download them.
6. Restart the app and confirm your notebook remains.

Run validation

```
pytest -q
python -m compileall app.py src scripts
```

2. Understand the implementation

SOURCE	PROCESS	STORE	ANSWER
PDF • PPTX • TXT • URL	Extract → clean → overlap chunks → embed	SQLite metadata + vector index	Retrieve → Gemini → citations → saved chat

File	Purpose
app.py	Starts the Gradio application
ui.py	Tabs, events, validation, and user feedback
ingestion.py	Reads files and webpages
text_utils.py	Cleans text and creates overlapping chunks
vector_store.py	Embeddings, persistent SQLite vectors, similarity and MMR
rag.py / llm.py	Grounded prompt, Gemini response, citations, timing
storage.py	SQLite notebooks, sources, chat, artifacts, evaluations
artifacts.py	Creates report.md and quiz.md
render.yaml / Dockerfile	Cloud runtime and persistent disk
ci-cd.yml	Automated tests, container build, deploy hook

The key RAG idea

The language model does not search your files directly. Your app finds relevant chunks first, then places those chunks in Gemini's prompt. Citation numbers map the answer back to the retrieved chunks.

3. Complete the required evaluation

The assignment requires two retrieval approaches. This implementation includes cosine similarity and maximum marginal relevance (MMR).

Method	How it chooses chunks	Tradeoff
Similarity	Highest semantic similarity to the question	Focused and usually faster; may repeat evidence
MMR	Balances relevance with difference from already selected chunks	Broader and less repetitive; extra computation

Evaluation procedure

1. Add representative course sources to one notebook.
2. Write at least five questions: direct fact, comparison, summary, multi-source synthesis, and an unanswerable question.
3. Run each question with both methods in the Evaluation tab.
4. Record retrieved chunk count, response time, citation accuracy (1–5), answer quality (1–5), and notes.
5. Complete docs/evaluation_template.md and select a default method using your evidence.

Do not invent measurements. The evaluator expects your actual test results and a short explanation of the observed tradeoff.

4. Push to GitHub

```
git init
git add .
git commit -m "Complete NotebookLM clone"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
git push -u origin main
```

If your GitHub repository already contains a commit, clone it first and copy these files into the cloned folder. Do not force-push unless you understand what will be replaced.

5. Deploy, record, and submit

Render deployment

1. In Render choose New → Blueprint, connect GitHub, and select the repository.
2. Approve the service and persistent disk defined in render.yaml.
3. Add GEMINI_API_KEY in Render's environment settings.
4. Deploy, open the public URL, and test every required feature.
5. Create a Render deploy hook and save it in GitHub Actions as RENDER_DEPLOY_HOOK_URL.
6. Push a small commit and verify the Actions run passes and deployment updates.

1–2 minute video

- Show the live URL and create/select a notebook.
- Upload a source and ask a grounded question.
- Point out citations and notebook-specific history.
- Generate and download a report and quiz.
- Show the similarity/MMR comparison and successful GitHub Actions run.

Final deliverables

Deliverable	Final check
GitHub repository	Public/accessible, clean README, no API key
Live application	Works in a private browser and persists data
Demo video	1–2 minutes, accessible link, no secrets
Architecture	docs/architecture.md matches the actual code
Evaluation	Real results and a justified method selection

Use SUBMISSION_CHECKLIST.md as your final gate. Replace every placeholder link in README.md before submission.